

BÁO CÁO THỰC HÀNH 01

OpenCV Basic - Xử lý ảnh cơ bản

Tên nhóm: _____
MSHV1: _____ MSHV2: _____

1 Mục tiêu

Bài thực hành xây dựng một Jupyter Notebook xử lý ảnh cơ bản. Các nội dung gồm đọc và hiển thị ảnh, scale, xoay ảnh, biến đổi màu sắc, độ sáng và độ tương phản. Notebook được chia lại theo yêu cầu: mỗi mục lớn có 3 code cell chính gồm cell thủ công, cell dùng thư viện và cell so sánh thời gian hoặc bộ nhớ.

2 Cấu trúc bài nộp

Gói nộp được tổ chức như sau:

- source/: chứa notebook, ảnh dự phòng, requirements.txt, environment.yml, README.md và thư mục kết quả.
- doc/: chứa báo cáo PDF, nguồn LaTeX và hình ảnh minh chứng.
- File nén cuối cùng theo mẫu TenNhom_MSHV1_MSHV2_th01.zip.

3 Ảnh đầu vào

Notebook có cell tải ảnh từ Internet bằng lệnh:

```
wget -q -O data/Lenna.jpg  
http://www.ess.ic.kanagawa-it.ac.jp/std_img/colorimage/Lenna.jpg
```

Nếu môi trường không có Internet, notebook tự tạo ảnh dự phòng để toàn bộ cell vẫn chạy được. Cách này giúp bài nộp không phụ thuộc tuyệt đối vào mạng trong lúc chấm.

4 Thư viện sử dụng

Thư viện	Vai trò
----------	---------

NumPy	Tự cài đặt các phép biến đổi trên ma trận pixel.
OpenCV	Đọc ảnh, ghi ảnh và dùng hàm thư viện để đối chiếu.
Matplotlib	Hiển thị ảnh trong notebook.
time	Đo thời gian chạy.
tracemalloc	Đo bộ nhớ Python cấp phát ở mức xấp xỉ.

5 Tổ chức notebook

Mỗi mục xử lý chính được chia theo đúng 3 cell:

1. **CELL THỦ CÔNG**: tự cài đặt bằng NumPy và thao tác pixel.
2. **CELL TỰ ĐỘNG / THƯ VIỆN**: dùng hàm OpenCV tương ứng.
3. **CELL SO SÁNH**: hiển thị ảnh, tính MSE, đo thời gian và bộ nhớ.

6 Minh chứng chức năng

6.1 Đọc và hiển thị ảnh

Phần thủ công tự chuyển BGR sang RGB bằng thao tác slicing. Phần thư viện dùng `cv2.cvtColor`. Vì OpenCV đọc ảnh theo thứ tự BGR còn Matplotlib hiển thị theo RGB, bước chuyển kênh là bắt buộc.



Figure 1: Minh chứng đọc ảnh và chuyển BGR sang RGB.

6.2 Scale ảnh

Phần thủ công cài đặt nội suy bilinear. Với mỗi pixel đầu ra, thuật toán ánh xạ về vị trí thực trên ảnh nguồn, lấy bốn pixel lân cận và nội suy theo hai chiều. Phần thư viện dùng `cv2.resize` với `INTER_LINEAR`.



Figure 2: So sánh scale thủ công và scale bằng OpenCV.

6.3 Xoay ảnh

Phần thủ công dùng ánh xạ ngược quanh tâm ảnh. Với mỗi pixel đầu ra, thuật toán tính ngược tọa độ nguồn rồi lấy pixel gần nhất. Phần thư viện dùng `cv2.getRotationMatrix2D` và `cv2.warpAffine`.

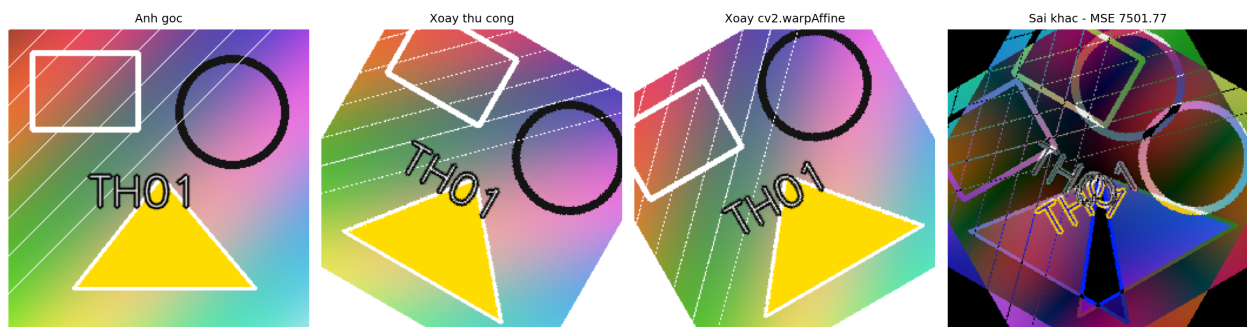


Figure 3: So sánh xoay ảnh thủ công và xoay bằng OpenCV.

6.4 Biến đổi màu sắc

Chuyển xám thủ công dùng công thức:

$$Y = 0.114B + 0.587G + 0.299R$$

Ảnh âm bản dùng công thức:

$$I' = 255 - I$$

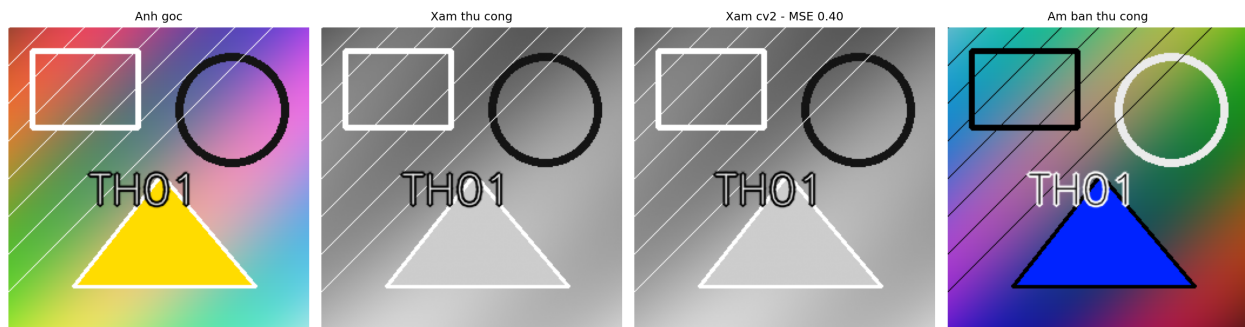


Figure 4: Minh chứng chuyển xám và tạo ảnh âm bản.

6.5 Độ sáng và độ tương phản

Phép biến đổi thủ công dùng công thức:

$$I' = \alpha I + \beta$$

Trong đó α điều chỉnh tương phản và β điều chỉnh độ sáng. Kết quả được chặn trong khoảng 0 đến 255. Phần thư viện dùng `cv2.convertScaleAbs`.



Figure 5: So sánh chỉnh sáng và tương phản thủ công với OpenCV.

7 So sánh và phân tích

Các kết quả thủ công và thư viện có thể không trùng tuyệt đối. Nguyên nhân thường nằm ở quy ước làm tròn, vị trí tâm pixel, cách xử lý biên và chi tiết nội suy trong OpenCV. Hàm thư viện thường nhanh hơn vì được tối ưu ở mức C/C++, trong khi cài đặt thủ công dùng vòng lặp Python nên chậm hơn nhưng giúp hiểu rõ bản chất thuật toán.

Bộ nhớ đo bằng `tracemalloc` chỉ phản ánh bộ nhớ Python cấp phát. Bộ nhớ nội bộ của OpenCV ở tầng C/C++ có thể không được ghi nhận đầy đủ, vì vậy số liệu memory nên dùng để tham khảo tương đối.

8 Bảng tự đánh giá

Yêu cầu	Trạng thái	Ghi chú
Đọc và hiển thị ảnh	Hoàn thành	Có cell wget, open và show.
Scale thay đổi kích thước	Hoàn thành	Có thủ công, thư viện và so sánh.
Xoay ảnh	Hoàn thành	Có ánh xạ ngược và warpAffine.
Biến đổi màu sắc	Hoàn thành	Có grayscale và âm bản.
Độ sáng, độ tương phản	Hoàn thành	Có công thức thủ công và OpenCV.
Cell thủ công	Hoàn thành	Mỗi mục lớn có cell riêng.
Cell tự động / thư viện	Hoàn thành	Dùng hàm OpenCV tương ứng.
Cell so sánh thời gian hoặc memory	Hoàn thành	Có thời gian, memory xấp xỉ, MSE và hình ảnh.
Có file đặc tả thư viện	Hoàn thành	Có requirements.txt và environment.yml.
Báo cáo riêng, không đưa mã nguồn	Hoàn thành	Báo cáo dùng mô tả, công thức, bảng và hình minh chứng.

9 Phân công đóng góp

Thành viên	Công việc	Tỉ lệ
MSHV1 - Họ tên: _____	Chuẩn bị notebook, phần open/show, scale, xoay và kiểm thử.	50%
MSHV2 - Họ tên: _____	Phần màu sắc, sáng/tương phản, báo cáo và tổng hợp hình minh chứng.	50%

10 Kết luận

Notebook đã được tổ chức theo đúng hướng yêu cầu: mỗi mục có cell thủ công, cell thư viện và cell so sánh. Cách trình bày này giúp việc chấm bài rõ ràng

hơn, đồng thời làm nổi bật sự khác biệt giữa tự cài đặt thuật toán và dùng hàm tối ưu sẵn có.